Working of MLP / ANN

A **Multilayer Perceptron (MLP)** is one of the most fundamental types of **Artificial Neural Networks (ANNs)**. It learns a relationship between input features and output predictions by repeatedly:

$$\text{Forward Propagation} \rightarrow \text{Loss Calculation} \rightarrow \text{Backpropagation} \rightarrow \text{Weight Update}$$

This process is repeated many times until the network learns a good mapping from inputs to outputs.

What is an ANN?

An **Artificial Neural Network** is a computational model inspired loosely by biological neurons.

A simple artificial neuron receives inputs, multiplies them by weights, adds a bias, and applies an activation function.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Then:

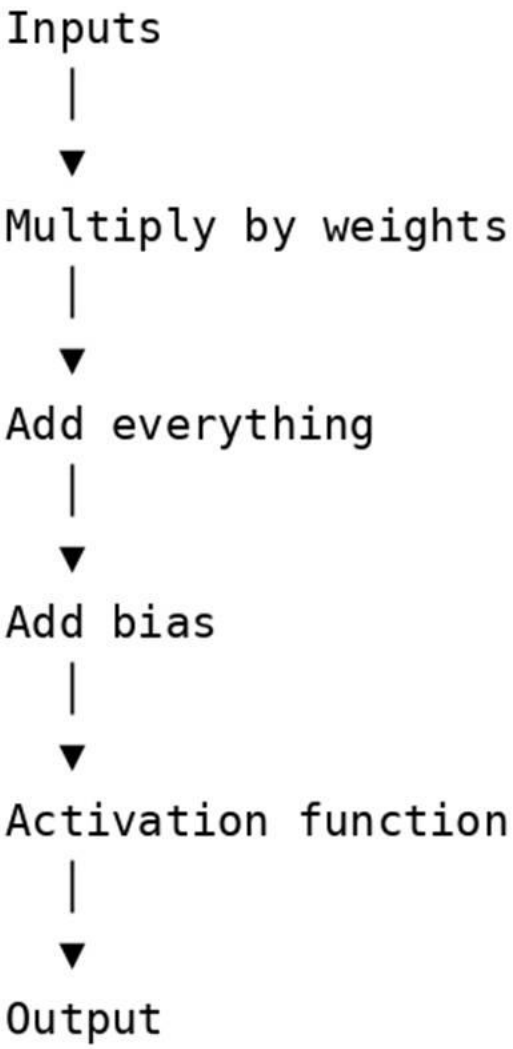
$$a = f(z)$$

where:

- x_i = input
- w_i = weight
- b = bias
- z = weighted sum

- f = activation function
- a = neuron's output

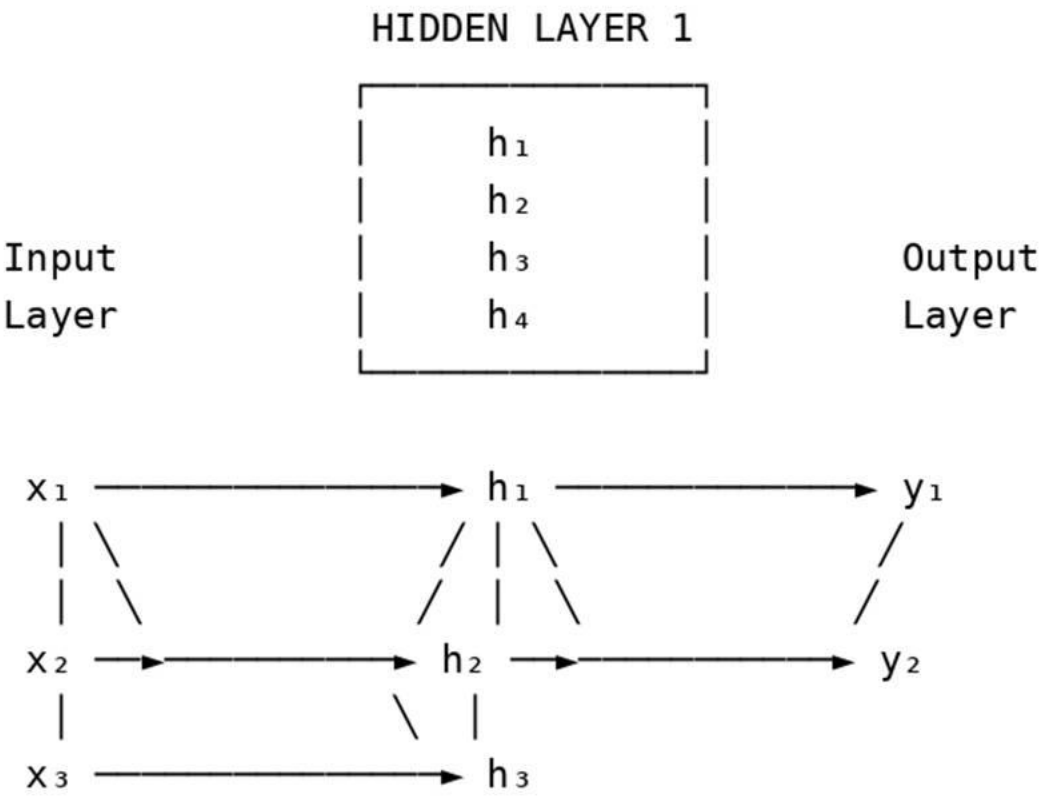
So the basic neuron performs:



What is an MLP?

An **MLP (Multilayer Perceptron)** is an ANN containing one or more **hidden layers**.

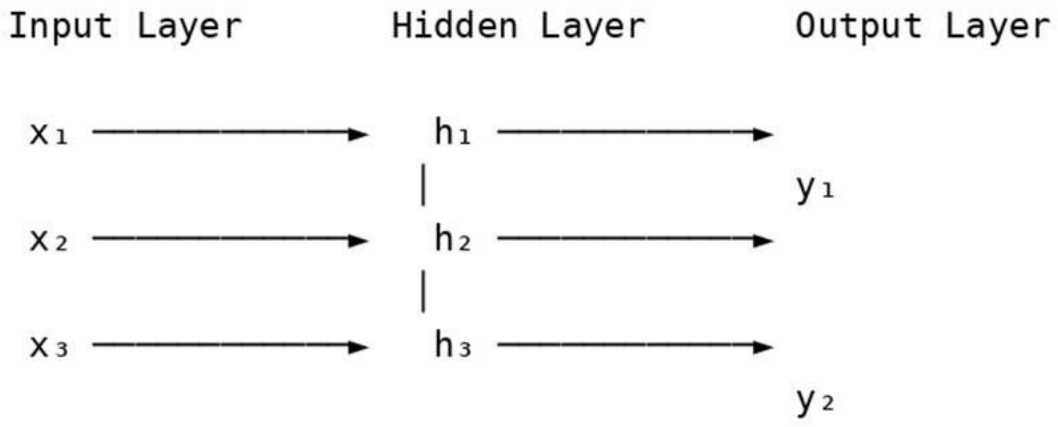
A typical MLP looks like:



More formally:

$$Input \rightarrow HiddenLayer(s) \rightarrow Output$$

For example:



Usually, every neuron in one layer is connected to every neuron in the next layer. This is why an MLP is also called a **fully connected neural network**.

The three major components of an MLP

An MLP generally contains:

1. Input layer

Receives the features.

2. Hidden layers

Learn patterns and representations from the input.

3. Output layer

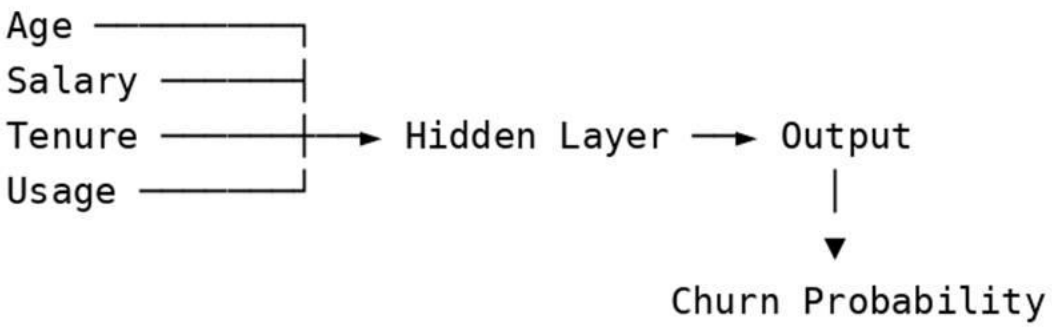
Produces the final prediction.

For example, suppose we want to predict whether a customer will leave a company.

Input features:

$$X = [Age, Salary, Tenure, Usage]$$

The network might be:



What exactly does a neuron do?

Suppose a neuron receives three inputs:

$$x_1, x_2, x_3$$

and corresponding weights:

$$w_1, w_2, w_3$$

The neuron calculates:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Then applies an activation:

$$a = f(z)$$

For example:

$$x_1 = 2, \quad x_2 = 3, \quad x_3 = 1$$

$$w_1 = 0.5, \quad w_2 = 0.2, \quad w_3 = -0.4$$

$$b = 0.1$$

Then:

$$z = (2)(0.5) + (3)(0.2) + (1)(-0.4) + 0.1$$

$$z = 1 + 0.6 - 0.4 + 0.1$$

$$z = 1.3$$

If ReLU is used:

$$a = ReLU(1.3) = 1.3$$

So the neuron outputs:

$$\boxed{1.3}$$

Why do we need weights?

Weights determine the **importance** of inputs.

Suppose:

$$z = w_1 x_1 + w_2 x_2$$

If:

$$w_1 = 10$$

and:

$$w_2 = 0.1$$

then x1 has much more influence on the neuron than x1.

Therefore:

Weights tell the network how important each input is for producing the output.

During training, these weights are learned automatically.

Why do we need bias?

Bias allows a neuron to shift its activation.

Without bias:

$$z = w_1 x_1 + w_2 x_2$$

With bias:

$$z = w_1 x_1 + w_2 x_2 + b$$

The bias gives the neuron additional flexibility.

A simple analogy:

- Weight controls the **slope/influence**
- Bias controls the **shift**

For a linear equation:

$$y = mx + c$$

the weight is similar to m, while bias is similar to c.

Why do we need activation functions?

This is extremely important.

Suppose we have multiple layers but **no activation functions**.

Layer 1:

$$Z_1 = XW_1 + b_1$$

Layer 2:

$$Z_2 = Z_1 W_2 + b_2$$

Substituting:

$$Z_2 = (XW_1 + b_1)W_2 + b_2$$

This can still be represented as essentially one linear transformation.

So simply adding many linear layers doesn't give the network powerful nonlinear learning capability.

Activation functions introduce:

$$\boxed{\text{Nonlinearity}}$$

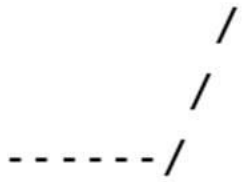
This allows neural networks to learn complex relationships.

Common activation functions

ReLU

$$ReLU(x) = \max(0, x)$$

Commonly used in hidden layers.



Advantages:

- Simple
- Computationally efficient
- Helps reduce vanishing gradients
- Very commonly used

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Output:

$$0 < \sigma(x) < 1$$

Often useful for binary output probabilities.

Tanh

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Output:

$$-1 < tanh(x) < 1$$

Softmax

Used commonly for multiclass classification.

For k classes:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

The outputs add up to 1:

$$\sum_i \hat{y}_i = 1$$

Example:

$$[0.1, 0.7, 0.2]$$

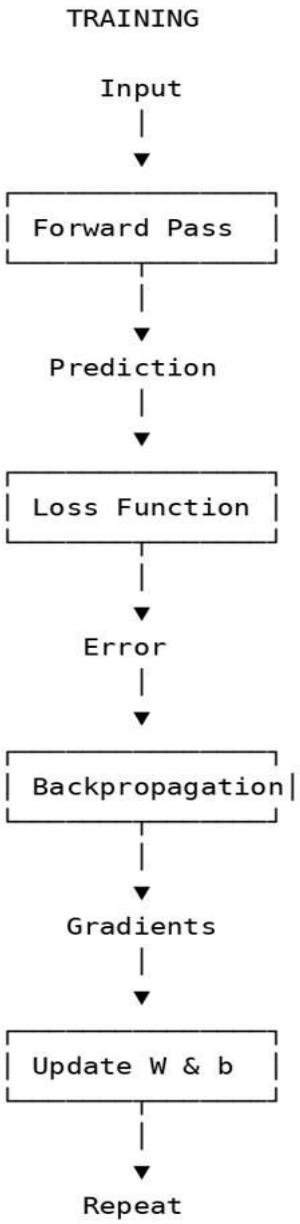
means approximately:

- Class 1 → 10%
- Class 2 → 70%

- Class 3 → 20%

Complete MLP working

The overall process is:



Let's understand every stage.

Step 1 — Input is given to the network

Suppose we have:

$$X = [x_1, x_2]$$

For example:

$$X = [1, 2]$$

These values enter the input layer.

The input layer generally does **not perform learning**.

It simply passes the feature values to the first trainable layer.

Step 2 — First hidden layer performs weighted sum

Suppose the first hidden layer has two neurons.

Weights:

$$W^{(1)} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

The matrix equation is:

$$\boxed{Z^{(1)} = XW^{(1)} + b^{(1)}}$$

For individual neurons:

$$z_1 = x_1 w_{11} + x_2 w_{21} + b_1$$

$$z_2 = x_1 w_{12} + x_2 w_{22} + b_2$$

These are called **pre-activation values** or **weighted sums**.

Step 3 — Activation is applied

Now:

$$A^{(1)} = f(Z^{(1)})$$

For ReLU:

$$A^{(1)} = ReLU(Z^{(1)})$$

For example:

$$Z^{(1)} = [-2, 3]$$

Then:

$$A^{(1)} = [0, 3]$$

because:

$$ReLU(-2) = 0$$

and:

$$ReLU(3) = 3$$

The activated values become the input to the next layer.

Step 4 — Information moves to the next layer

Suppose:

$$A^{(1)} = [0, 3]$$

The next layer calculates:

$$Z^{(2)} = A^{(1)} W^{(2)} + b^{(2)}$$

Again:

$$\text{weighted sum} \rightarrow \text{activation}$$

This continues through all hidden layers.
For a deeper network:

$$X \rightarrow Z^{(1)} \rightarrow A^{(1)} \rightarrow Z^{(2)} \rightarrow A^{(2)} \rightarrow Z^{(3)} \rightarrow A^{(3)}$$

and so on.

Step 5 — Output layer produces prediction

The output layer depends on the problem.

Regression

Usually:

$$\hat{y} = Z$$

or a linear activation.

Example:

Predict house price:

$$\hat{y} = ₹65,00,000$$

Binary classification

Usually:

$$\hat{y} = \sigma(Z)$$

Example:

$$\hat{y} = 0.82$$

Interpretation:

$$82\%$$

probability of the positive class.

Multiclass classification

Usually:

$$\hat{Y} = Softmax(Z)$$

Example:

$$[0.05, 0.15, 0.80]$$

The model predicts class 3.

Step 6 — Calculate loss

The prediction is compared with the actual target.

Suppose:

$$Y = [1, 0]$$

and prediction:

$$\hat{Y} = [0.7, 0.3]$$

The model is somewhat correct but not perfect.

The loss function measures how wrong the model is.

For classification:

$$L = -\sum_{i} y_i \log(\hat{y}_i)$$

For this example:

$$L = -[1\log(0.7) + 0\log(0.3)]$$

$$L = -\log(0.7)$$

$$\boxed{L \approx 0.357}$$

Lower loss generally means a prediction closer to the target.

Step 7 — Backpropagation

Backpropagation means:

Propagating the error gradient backward through the network.

Forward propagation:

$$\boxed{Input \rightarrow Hidden \rightarrow Output}$$

Backpropagation:

$$\boxed{Loss \rightarrow Output \rightarrow Hidden \rightarrow Input}$$

The important point is that **we don't propagate the original error itself backward**.

We propagate **gradients**.

What is a gradient?

A gradient tells us:

How much the loss changes when a parameter changes slightly.

For a weight w :

$$\frac{\partial L}{\partial w}$$

means:

How much does the loss change when w changes?

For example:

$$\frac{\partial L}{\partial w} = 0.8$$

means increasing that weight slightly tends to increase the loss.

If:

$$\frac{\partial L}{\partial w} = -0.8$$

increasing that weight tends to decrease the loss.

The chain rule is the heart of backpropagation

Consider:

$$x \to z \to a \to L$$

where:

$$z = wx + b$$

$$a = f(z)$$

$$L = L(a)$$

We want:

$$\frac{\partial L}{\partial w}$$

Using the chain rule:

$$\boxed{\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w}}$$

Since:

$$z = wx + b$$

we have:

$$\frac{\partial z}{\partial w} = x$$

Therefore:

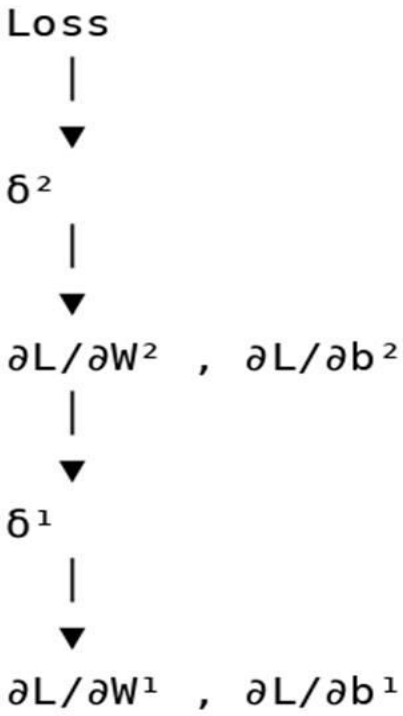
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}x$$

This is the basic idea behind backpropagation.

Backpropagation through an entire MLP

For a two-layer MLP:

Backward:



For Softmax + Cross-Entropy:

$$\delta^{(2)} = \hat{Y} - Y$$

Then:

$$\frac{\partial L}{\partial W^{(2)}} = (A^{(1)})^T \delta^{(2)}$$

and:

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

Then the error is propagated to the hidden layer:

$$\delta^{(1)} = \left[\delta^{(2)} (W^{(2)})^T\right] \odot f'(Z^{(1)})$$

Then:

$$\frac{\partial L}{\partial W^{(1)}} = X^T \delta^{(1)}$$

and:

$$\frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

Step 8 — Update weights

Once gradients have been calculated, the network updates its parameters.
The basic gradient descent equation is:

$$W_{new} = W_{old} - \eta \nabla W$$

Similarly:

$$b_{new} = b_{old} - \eta \nabla b$$

where:

- W = weight
- b = bias
- ∇W = gradient of loss with respect to weight
- ∇b = gradient of loss with respect to bias
- η = learning rate

Why do we subtract the gradient?

Suppose:

$$w = 2$$

and:

$$\frac{\partial L}{\partial w} = 0.5$$

Learning rate:

$$\eta = 0.1$$

Then:

$$w_{new} = 2 - (0.1)(0.5)$$

$$\boxed{w_{new} = 1.95}$$

Why subtract?

Because the gradient points in the direction of **increasing loss**.

We want to move in the opposite direction:

$$\boxed{\text{Direction of decreasing loss}}$$

Learning rate

The learning rate controls how large the update is.

$$W_{new} = W - \eta \nabla W$$

Small learning rate

Example:

$$\eta = 0.0001$$

Updates are very small.

Training can be very slow.

Large learning rate

Example:

$$\eta = 1$$

Updates may be too large.

The model may:

- overshoot the minimum
- oscillate
- become unstable
- fail to converge

A commonly used starting point for Adam is often:

$$\eta = 0.001$$

but the appropriate value depends on the problem.

One complete training iteration

Let's put everything together.

Suppose:

$$X$$

is the input.

Forward

$$Z^{(1)} = XW^{(1)} + b^{(1)}$$

$$\downarrow$$

$$A^{(1)} = f(Z^{(1)})$$

$$\downarrow$$

$$Z^{(2)} = A^{(1)}W^{(2)} + b^{(2)}$$

$$\downarrow$$

$$\hat{Y} = g(Z^{(2)})$$

$$\downarrow$$

Loss

$$L(Y, \hat{Y})$$

$$\downarrow$$

Backward

$$\frac{\partial L}{\partial W^{(2)}}$$

$$\frac{\partial L}{\partial b^{(2)}}$$

$$\frac{\partial L}{\partial W^{(1)}}$$

$$\frac{\partial L}{\partial b^{(1)}}$$

$$\downarrow$$

Update

$$W^{(1)} \leftarrow W^{(1)} - \eta \nabla W^{(1)}$$

$$b^{(1)} \leftarrow b^{(1)} - \eta \nabla b^{(1)}$$

$$W^{(2)} \leftarrow W^{(2)} - \eta \nabla W^{(2)}$$

$$b^{(2)} \leftarrow b^{(2)} - \eta \nabla b^{(2)}$$

That is **one training step**.

Training step vs batch vs epoch

These terms are very important.

Suppose we have:

$$50,000$$

training examples.

Batch size:

$$128$$

Then approximately:

$$\frac{50,000}{128} \approx 391$$

batches are processed in one epoch.

One batch

The model:

1. Takes 128 samples
2. Performs forward propagation
3. Calculates loss
4. Performs backpropagation
5. Updates weights

That's one **training step/iteration**.

One epoch

After all 391 batches have been processed:

$$\boxed{\text{1 epoch completed}}$$

If:

$$epochs = 15$$

the model goes through the training dataset approximately 15 times.

Important distinction: batch size vs epoch

Suppose:

```
Training samples = 50,000
Batch size = 100
Epochs = 10
```

Then:

$$500$$

steps per epoch.
Therefore:

$$500 \times 10 = 5000$$

weight-update steps approximately.
So the weights are **not updated once per epoch**.
They are updated **once per batch**.

What happens during an epoch?

Conceptually:

```
Epoch 1
  |
  ├── Batch 1 → Forward → Loss → Backward → Update
  ├── Batch 2 → Forward → Loss → Backward → Update
  ├── Batch 3 → Forward → Loss → Backward → Update
  ├── ...
  └── Batch 391 → Forward → Loss → Backward → Update
                              |
                              ▼
                        Epoch complete

Epoch 2
  |
  ├── Batch 1
  ├── Batch 2
  ├── ...
  └── Batch 391

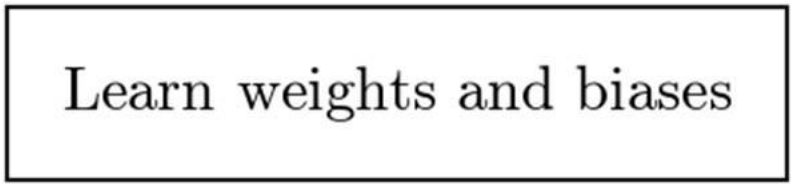
...
```

Training vs validation vs testing

An MLP normally works with three datasets.

Training set

Used to:



Validation set

Used to:

Monitor performance during training

It is useful for:

- hyperparameter selection
- detecting overfitting
- model selection
- early stopping

Test set

Used after training to estimate:

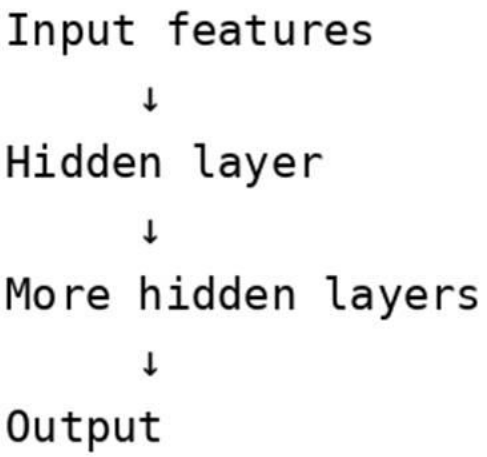
Final generalization performance

The test set should not be used to update weights.

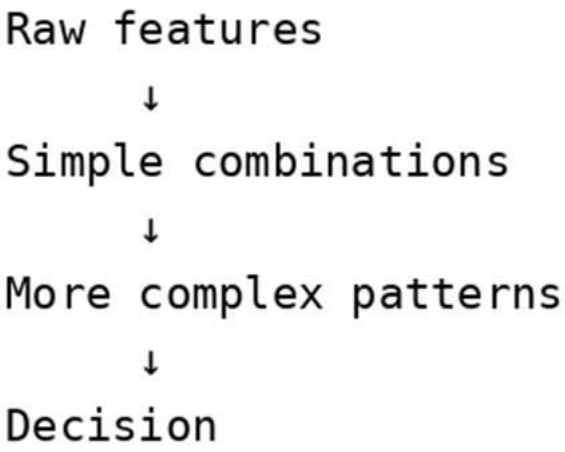
What does the network actually learn?

This is a very important conceptual question.

Suppose:



The network progressively learns representations.
For a simple problem:



For example, with tabular data:

$$Age, Salary, Experience$$

a hidden neuron might learn something similar to:

$$0.5(Age) + 0.8(Salary) - 0.2(Experience)$$

Another neuron learns a different combination.

The network combines these learned representations to make the final prediction.

Why multiple hidden layers?

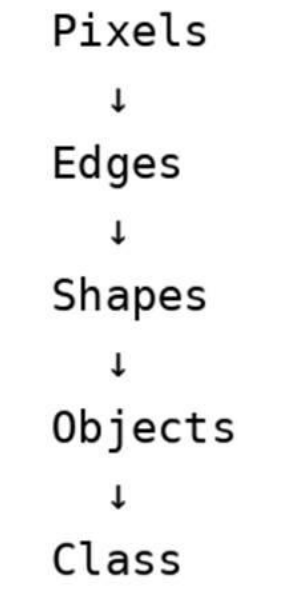
One hidden layer can theoretically approximate many functions, given enough neurons.

But deep networks use multiple layers because they can learn hierarchical representations.

Conceptually:

$$X \rightarrow \text{simple patterns} \rightarrow \text{intermediate patterns} \rightarrow \text{complex patterns} \rightarrow \text{output}$$

For example, in an image-related task:



In an MLP on tabular data, the learned representations are not necessarily as visually interpretable, but the same hierarchical idea applies.

Why initialization matters

At the beginning:

$$W$$

must be initialized.

We generally don't want all weights to be identical.

If all neurons start with exactly the same weights, they tend to produce the same outputs and receive similar gradients.

This prevents useful **symmetry breaking**.

Common initialization methods include:

Xavier / Glorot

Often associated with:

- sigmoid
- tanh

He / Kaiming

Commonly associated with:

- ReLU
- Leaky ReLU

For example, He initialization is designed to maintain appropriate activation/gradient variance for ReLU-like networks.

Why normalization is often important

Suppose one feature is:

$$Age = 25$$

and another is:

$$Salary = 500000$$

Their scales are dramatically different.

Neural networks generally train more effectively when input features have reasonably controlled scales.

For images, a common normalization is:

$$\frac{x}{255}$$

so pixel values:

$$0 - 255$$

become:

$$0 - 1$$

This often makes optimization easier.

What is overfitting?

Suppose the model performs extremely well on training data:

$$TrainingAccuracy = 99\%$$

but poorly on unseen data:

$$ValidationAccuracy = 80\%$$

This may indicate:

Overfitting

The network has learned the training data too specifically instead of learning patterns that generalize.

Common techniques to reduce overfitting include:

- Dropout
- L1/L2 regularization
- Early stopping
- Data augmentation where applicable
- More training data
- Reducing model complexity

What is underfitting?

Suppose:

$$Training Accuracy = 70\%$$

and:

$$Validation Accuracy = 68\%$$

The model may not have learned the underlying relationship sufficiently.

Possible causes:

- model too small
- insufficient training
- inappropriate features
- excessive regularization
- poor optimization

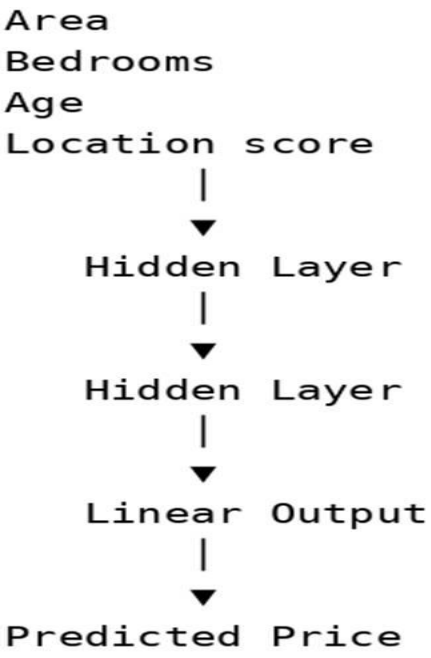
35. MLP with different output problems

The architecture changes depending on the problem.

Problem	Output layer	Common loss
Regression	Linear	MSE / MAE
Binary classification	Sigmoid	Binary Cross Entropy
Multiclass classification	Softmax	Categorical Cross Entropy
Multiclass with integer labels	Softmax	Sparse Categorical Cross Entropy
Multi-label classification	Multiple sigmoid outputs	Binary Cross Entropy

Example: regression MLP

Suppose we want to predict house price.



Output:

$$\hat{y}=₹72,50,000$$

Actual:

$$y=₹75,00,000$$

MSE could be:

$$MSE=\frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2$$

37. Example: binary classification MLP

Suppose we want to predict whether a customer will churn.
Output:

$$\hat{y}=0.82$$

Using sigmoid:

$$0\leq\hat{y}\leq1$$

We might interpret:

$$82\%$$

probability of class 1.

Example: multiclass classification

Suppose we classify an image into:

```
Cat
Dog
Horse
```

Output:

$$\hat{Y} = [0.10, 0.75, 0.15]$$

The largest probability is:

$$0.75$$

Therefore:

$$\boxed{\text{Prediction} = \text{Dog}}$$

Where does an optimizer such as Adam fit?

The basic training pipeline is:

$$\boxed{Forward \rightarrow Loss \rightarrow Backpropagation \rightarrow Optimizer \rightarrow Update}$$

Backpropagation calculates the gradients.

The optimizer determines how those gradients are used to update parameters.

For simple gradient descent:

$$W_{t+1} = W_t - \eta \nabla W_t$$

For Adam, the update is more sophisticated and uses running estimates of the first and second moments of the gradients.

So:

Backpropagation calculates gradients; the optimizer uses those gradients to update parameters.

This distinction is very important.

Forward vs backward propagation

Forward Propagation	Backward Propagation
Moves input → output	Moves loss → earlier layers
Calculates predictions	Calculates gradients
Uses weights and biases	Uses gradients of loss
Uses activation functions	Uses activation derivatives
Ends with loss calculation	Starts from loss
Answers "What did we predict?"	Answers "How should we change?"

A very useful way to remember it:

$$\boxed{\text{Forward} = \text{Prediction}}$$

$$\boxed{\text{Backward} = \text{Learning}}$$

The key idea behind neural-network learning

An MLP does **not** explicitly learn rules such as:

"If Age > 40 and Income > X, predict class 1."

Instead, it starts with parameters:

$$W, b$$

and gradually learns values for them.

Training is essentially an optimization problem:

$$\text{Find } W, b \text{ that minimize } L(W, b)$$

So we can think of neural-network training as:

$$\text{Learning} = \text{Finding good weights and biases}$$

One complete example model

If you're teaching or learning MLPs, remember this sequence:

Step 1 — Input

$$X$$

"What information did I receive?"

Step 2 — Weighted sum

$$Z = XW + b$$

"What does each neuron calculate from the inputs?"

Step 3 — Activation

$$A = f(Z)$$

"What nonlinear pattern does this neuron represent?"

Step 4 — Prediction

$$\hat{Y}$$

"What does the network think?"

Step 5 — Loss

$$L(Y, \hat{Y})$$

"How wrong is the network?"

Step 6 — Backpropagation

$$\nabla W, \nabla b$$

"Which parameters caused the error, and by how much?"

Step 7 — Optimization

$$W \leftarrow W - \eta \nabla W$$

"How should the parameters change?"

Step 8 — Repeat

$$\text{Repeat for many batches and epochs}$$

"Keep learning until the model performs sufficiently well."

45. Final summary

The complete working of an MLP can be represented by:

$$X \xrightarrow{\text{Forward}} Z^{(1)} \to A^{(1)} \to Z^{(2)} \to \hat{Y} \to L$$

Then:

$$L \xrightarrow{\text{Backpropagation}} \nabla W^{(2)}, \nabla b^{(2)} \to \nabla W^{(1)}, \nabla b^{(1)}$$

Then:

$$W, b \xrightarrow{\text{Optimizer}} W_{new}, b_{new}$$

and the cycle repeats:

$$\text{Forward} \to \text{Loss} \to \text{Backward} \to \text{Update} \to \text{Repeat}$$